

1. Táblázat sorának hozzáadása

A táblázatnál a trükk az, hogy a sor (tr) és a cella (td) egymásba ágyazódik.

```
// 1. Meglévő táblázat kijelölése
let tabla = document.querySelector("table");

// 2. Új sor (tr) létrehozása a táblázat végére
let ujSor = tabla.insertRow();

// 3. Új cellák (td) hozzáadása a sorhoz
let cella1 = ujSor.insertCell(0);
let cella2 = ujSor.insertCell(1);

// 4. Tartalom beleírása
cella1.innerText = "Első oszlop";
cella2.innerText = "Második oszlop";
```

1. Táblázat (Thead és Tbody használatával)

A) Meglévő elemhez hozzáadás (csak egy új adatsor):

```
// Feltételezzük, hogy a HTML-ben már van egy <tbody id="adat-torzs">
let tbody = document.getElementById("adat-torzs");

let ujSor = tbody.insertRow(); // Létrehozza a <tr>-t a tbody-n belül
let cella1 = ujSor.insertCell(0); // Létrehozza az első <td>-t
let cella2 = ujSor.insertCell(1); // Létrehozza a második <td>-t

cella1.innerText = "01";
cella2.innerText = "Kovács János";
```

B) Táblázat létrehozása a nulláról (szerkezettel együtt):

```
// 1. Fő elemek létrehozása
let tabla = document.createElement("table");
let thead = document.createElement("thead");
let tbody = document.createElement("tbody");

// 2. Fejléc sor és cellák (manuálisan)
let fejlécSor = document.createElement("tr");
let th1 = document.createElement("th");
th1.innerText = "ID";
let th2 = document.createElement("th");
th2.innerText = "Név";
```

```
fejlecSor.appendChild(th1);
fejlecSor.appendChild(th2);
thead.appendChild(fejlecSor);

// 3. Egy kezdő adatsor hozzáadása a tbody-hoz
let adatSor = tbody.insertRow();
adatSor.insertCell(0).innerText = "1";
adatSor.insertCell(1).innerText = "Teszt Elek";

// 4. Összeszerelés: a table-be belepakoljuk a részeket
tabla.appendChild(thead);
tabla.appendChild(tbody);
// 5. Kitevés az oldalra
document.body.appendChild(tabla);
```

2. Lista (Ordered List)

A) Meglévő listához hozzáadás:

```
let lista = document.getElementById("sorszamozott-lista");
let li = document.createElement("li");
li.innerText = "Új elem a listában";
lista.appendChild(li);
```

B) Lista létrehozása a nulláról:

```
let ujLista = document.createElement("ol");
let li1 = document.createElement("li");
li1.innerText = "Első tétel";
let li2 = document.createElement("li");
li2.innerText = "Második tétel";

ujLista.appendChild(li1);
ujLista.appendChild(li2);
document.body.appendChild(ujLista);
```

3. Szöveges mezők (TextFields)

A) Meglévő div-be beszúrás:

```
let kontener = document.getElementById("mezo-kontener");
let input = document.createElement("input");
input.type = "text";
input.placeholder = "Írj ide...";

kontener.appendChild(input);
```

B) Mező létrehozása a nulláról (címkével együtt):

```
let mezoCsoport = document.createElement("div");
```

```
let label = document.createElement("label");  
label.innerText = "Email cím: ";
```

```
let emailInput = document.createElement("input");  
emailInput.type = "email";
```

```
mezoCsoport.appendChild(label);  
mezoCsoport.appendChild(emailInput);  
document.body.appendChild(mezoCsoport);
```

Mi a tanulság?

- Ha **meglévő** elemhez adsz hozzá, a legfontosabb, hogy pontosan "célozd meg" a szülőt (pl. ne a table-höz, hanem a tbody-hoz adj sort).
- Ha **nulláról** építkezik a tanuló, mindig **belülről kifelé** vagy **fentről lefelé** haladjon: hozza létre a legkisebb egységet (td/li), tegye bele a közvetlen szülőjébe (tr/ol), és a legvégén tegye ki a főelemet az oldalra.

A stílusok hozzáadása a dinamikus elemekhez olyan, mintha felöltöztetnéd az elemeket, mielőtt kiküldöd őket az oldalra. Erre a legprofibb módszer a classList használata.

Íme a kiegészítés az előzőekhez:

1. Táblázat stílusozása

Amikor nulláról hozod létre, rögtön adj neki egy osztályt, amit a CSS-ben már megírtál.

```
let tabla = document.createElement("table");  
tabla.classList.add("modern-table"); // CSS-ben: .modern-table { ... }
```

// A soroknál is működik (pl. kiemelés)

```
let sor = tabla.insertRow();  
sor.classList.add("highlight-row");
```

2. Lista stílusozása

A listaelemeknél gyakran kell egyedi kinézet, például ikonok vagy margók miatt.

```
let li = document.createElement("li");  
li.innerText = "Fontos feladat";  
li.classList.add("list-item-styled");
```

// Akár több osztályt is kaphat:

```
li.classList.add("active", "priority-high");
```

3. Szövegmezők (Input) csinosítása

Az inputoknál különösen fontos a stílus, hogy ne a gyári, "unalmas" kinézetet kapják.

```
let mezo = document.createElement("input");  
mezo.type = "text";  
mezo.classList.add("form-control", "shadow-sm"); // Bootstrap-szerű osztályok
```

Miért a `classList.add` a legjobb?

- **Tiszta kód:** A stílusok a CSS fájlban maradnak, a JS csak "ráaggatja" a címkét az elemre.
- **Könnyű módosítás:** Ha megváltozik a design, csak a CSS-t kell átírnod, a JavaScripthez hozzá sem kell nyúlnod.
- **Több osztály:** Egyszerre több tulajdonságot is adhatsz neki (pl. legyen kék ÉS legyen félkövér).

Összefoglaló táblázat a tanulóknak

Funkció	JavaScript kód	Mi történik?
Osztály hozzáadása	<code>elem.classList.add("nev")</code>	Hozzáad egy CSS osztályt
Osztály levétele	<code>elem.classList.remove("nev")</code>	Törli az osztályt az elemről
Váltogatás (Toggle)	<code>elem.classList.toggle("nev")</code>	Ha rajta van leveszi, ha nincs, ráteszi